

REN -206 - Working with Databases

Introduction

In this module, we will introduce the various ways that you can work with databases from your Rhapsody configuration. We will focus primarily on the Database communication point, as it offers the largest set of features when working with a database.

Throughout the module, we will use the following scenarios to illustrate the range of tasks that can be performed:

- Inserting a record into the database
- Updating the message from a database
- Polling a database looking for new records
- Replacing a message with the contents of a database query

Components

Rhapsody includes several Communication Points and Filters for working with databases.

Database Communication Point

Supports read and write access to the database to run arbitrary queries against the database. The communication point supports Output, Out->In, and Input modes. The Database Configuration Editor is used to configure this communication point.

Database Communication Point

Database Insertion Communication Point

Inserts or updates a single row in an existing database table. This communication point can only be run in Output mode. The configuration for this communication point is much simpler than the Database communication point.

Database Insertion Communication Point

Database LookUp Filter

This filter behaves identically to the Database communication point running in Out->In mode. The Database Configuration Editor is used to configure the filter.

Database LookUp Filter

① It is recommended that the Database Communication Point is used in preference to this filter as communication points include better management of connection issues.

Database Message Extraction Filter

Uses the existing message to execute one or more data queries. The filter replaces the existing message body with an XML representation of the queries run by the filter.

The Database Configuration Editor is used to configure this filter.

While we will discuss all the components above in this module, we will be focusing on the Database Communication Point as this is the most commonly used component for database interactions.

Database Drivers

All database communication points and filters use a Java [JDBC driver\(opens in a new tab\)](#) to communicate with the database server. Each JDBC driver is specific to the database you wish to connect to. By default, Rhapsody provides drivers to connect with the following databases:

- Microsoft SQL Server (using either the jTDS or Microsoft database driver)
- Oracle (using the oraThin database driver)
- MySQL
- Microsoft Access (using the JDBC-ODBC bridge driver)
- PostgreSQL

For all database connections, it is possible to specify basic connection details (hostname, port, username, & password) along with any database specific settings (such as fail-over or connection encryption settings).

Using Other Database Drivers

Where one of the default drivers cannot be used, it is possible to manually configure Rhapsody to connect to any database that has a working JDBC driver. Information on the appropriate format of the JDBC connection string for the various database types, along with details on connecting to other database types, can be found in the Rhapsody Documentation.

View further information in the documentation.

[Open docs](#)

⚠ When using a custom database driver, filter testing will return a database driver error. This is expected.

A custom database driver will be installed and configured as part of the exercises for this module.

Best Practices

Database configurations can be complex. It is therefore wise to make sure you incorporate best practices into the development of your configuration.

- Use Rhapsody Variables in the database setup for server name, database, user and port number. This makes it easier to migrate your configuration between environments where different database connection settings will likely be needed.
- Working with databases requires some planning. You are accessing data in a system outside of Rhapsody and need to understand the structure (including indexing) before making any modifications.
- When accessing a database belonging to another application, remember that it will have generally been tuned for use by their main application, not secondary usage by applications such as Rhapsody. Your configuration may impact the performance and stability of the primary application.
- Database access can be resource intensive and time-consuming. Consider monitoring performance while under load.
- Get a Database Administrator to review your queries and stored procedures to ensure that you are accessing the database efficiently. Poorly written queries can make some database tasks orders of magnitude slower.

SQL Formatting

SQL is just like any other language (such as JavaScript or Mapper) you include as part of your Rhapsody configuration. A set of best practices will go a long way to making the code easier to develop and maintain. It will also make the code more resilient to errors, changes to the underlying database structure and unexpected message inputs.

Like all best practices lists, they tend to be subjective, with individual lists built from hard learned experience and words of wisdom from those more knowledgeable. It is not the intention of this content to force a single SQL "style" on everyone, rather the purpose is to highlight the importance of a good SQL style and best practices. Where a

site has an existing set of best practice guidelines, they should be followed to ensure consistency of code.

Suggested Best Practice

The suggested best practices below represent items that appear in most SQL best practice discussions.

Recommendation

In a SELECT statement, the SELECT, FROM, WHERE and ORDER BY keywords should appear at the start of a line. In an UPDATE statement, the UPDATE and WHERE keywords should appear on newlines. In an INSERT statement, the INSERT and VALUES keywords should appear on a newline.

All other parts of the SQL statement should be appropriately indented, with each part of the statement separated by a newline.

Reason

This structure highlights the start of each new part of the statement, making the complete statement easier to follow. The use of newlines makes it easier to identify individual parts of the statement. Where the statement is quite large, this also helps prevent parts of the statement from running off the right of the screen, which makes it hard for a user to review the code.

Recommendation

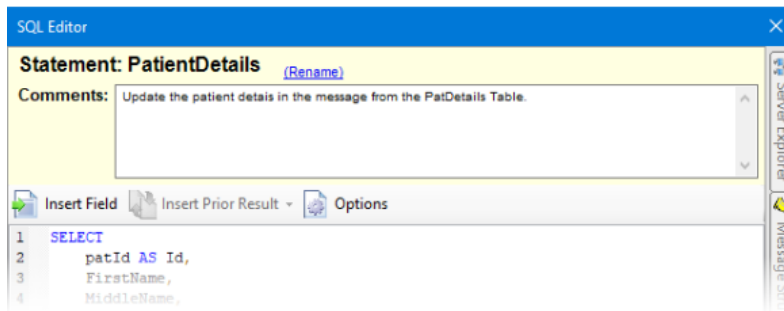
Include comments with the SQL statements.

When using the database editor in Rhapsody, these can be added alongside the query. Where understanding requires comments to be added inline, the -- comment syntax is used for single line comments, while /* comment */ syntax is used for multi-line comments.

Reason

Comments help explain what a SQL statement is doing and assist others in understanding the purpose of a statement.

When viewing the list of database statements associated with a database communication point or filter, the statement names and descriptions help explain the SQL without needing to review each statement.



Recommendation

SQL keywords (SELECT, FROM, WHERE, AS, AND, OR, ...) should appear in CAPS, while all other text should either be in lower case or mixed case.

Reason

This makes it easier to identify the keywords in a SQL statement when quickly viewed.

Recommendation

Use table aliases.

Reason

This makes the SQL statement easier to read, especially where multiple tables are utilized in the statement. Use of aliases also has a small performance gain as the database server is more easily able to determine the column required in the statement.

Recommendation

When selecting fields, always specify the column names. Never use select all (SELECT *) to select all rows in a table.

Reason

The select all syntax does not reveal what columns in the database are important. This can result in the database server doing more work to return the result set. It also makes it harder for a developer to identify the important fields in the result set during any modifications to the query or database.

Recommendation

Where there are joins in the query to additional tables, INNER JOIN and OUTER JOIN clauses should be used to link the tables (referred to as ANSI Joins). The old style that mixed the join logic as part of the WHERE clause should be avoided.

Reason

This makes the SQL statement more readable, as criteria that join tables are clearly separated from criteria that filter the result set.

Example 1

In the first example, a simple SQL SELECT statement is presented with no meaningful formatting. Reviewing this SQL statement will take more effort than the identical statement presented with formatting.

```
select report.id, report.username, datetime, report.description from report where  
id = @ObservationMessage[0].Order.OBR.UniversalServiceID.Identifier AND report.site  
= 'LIS'
```

The following SQL statement is functionally identical to the statement above. However, good formatting makes it much easier to understand.

```
SELECT report.id,  
       report.username,  
       report.datetime,  
       report.description  
FROM report  
WHERE report.id = @ObservationMessage[0].Order.OBR.UniversalServiceID.Identifier  
       AND report.site = 'LIS'
```

Example 2

The following UPDATE statement makes good use of indents and newlines to make the intent of the query easy to follow.

```
UPDATE patient  
    SET lastAccessTime = @MSH.DateTimeOfMessage.Time  
WHERE patientId =  
    @ObservationMessage[0].PatientInformation.PID.PatientIdentifierList[0].IDNumber  
    AND patientIdType =  
    @ObservationMessage[0].PatientInformation.PID.PatientIdentifierList[0].IdentifierType  
Code  
    AND active = true
```

Example 3

The following statement uses both SELECT all and non-ASCII join syntax. This makes it hard to determine which of the fields returned by the statement are important and

obscures the logic used to join the two tables. The lack of table aliases makes it hard to determine which table is being referred in each WHERE clause.

```
SELECT
```

```
    *
```

```
FROM patient, patAddress
```

```
WHERE patient.id = patAddress.patientId
```

```
    AND patientId =
```

```
@ObservationMessage[0].PatientInformation.PID.PatientIdentifierList[0].IDNumber
```

```
    AND patientIdType =
```

```
@ObservationMessage[0].PatientInformation.PID.PatientIdentifierList[0].IdentifierType  
Code
```

```
    AND current = true
```

In this reworked query, it is immediately clear what information the query is returning. The table join logic is also easy to spot and decipher.

```
SELECT
```

```
    patient.givenName,
```

```
    patient.middleName,
```

```
    patient.familyName,
```

```
    address.city,
```

```
    address.postCode
```

```
FROM patient
```

```
    INNER JOIN patAddress AS address ON patient.id = address.patientId
```

```
WHERE patient.patientId =
```

```
@ObservationMessage[0].PatientInformation.PID.PatientIdentifierList[0].IDNumber
```

```
    AND patient.patientIdType =
```

```
@ObservationMessage[0].PatientInformation.PID.PatientIdentifierList[0].IdentifierType  
Code
```

```
    AND address.current = true
```

Further Reading

- <http://www.codeproject.com/Articles/126380/Writing-Readable-SQL>(opens in a new tab)
- <http://weblogs.sqlteam.com/jamesw/archive/2011/10/03/a-list-of-sql-best-practices.aspx>(opens in a new tab)
- <http://sqlformat.org/>

Process

Tasks

The various database components in Rhapsody can be used to perform a variety of tasks. In the table below, we introduce the common database tasks that you might be expected to perform and the components that can be used to perform them.

Task	Details	Component Options
Updating a database	When updating a database, the contents of the message or fields from the message and message properties are used to populate one or more tables in a database.	Database communication point in Output mode or Database Insertion communication point
	① When updating a database, the Database communication point will be more often used as it implements a more complete solution for writing records to a database.	
Updating a message	We can update fields in a message and message properties with the results of one or more database queries. At the same time that the message is updated, it is also possible for the message to be used to update fields in the database.	Database communication point in Out->In mode or Database LookUp filter
	① The Database communication point in Out->In mode and the Database LookUp filter operate the same way. However, it is recommended that the communication point is used over the filter as it makes use of all the standard communication point connection management features. This allows better connection failure and retry behaviors to be implemented.	
Polling a database for a message	When polling a database, we are looking for records that have been written into it by an external system. These records can be queried to generate XML messages in Rhapsody. When polling a database to retrieve records, there should to be a column in the database table that can be used to filter new messages in your SQL. This column will often contain a date or a sequence number.	Database communication point in Input mode
Replacing the message body	When many fields need to be retrieved from a database, we can use the contents of the message to run one or more queries that replace the message body with an XML message. The format of the XML message is identical to the messages generated when polling the database.	Database Message Extraction filter

Depending on the component you are using and the tasks being performed, you will sometimes be able to combine multiple tasks (for example, updating a database and updating a message) into a single component.

Configuration Process

The process for creating a new database configuration when using the Database communication point, Database LookUp filter, and Database Message Extraction filter components follows the same steps regardless of the task being performed. This can be summarized as:

1. 1

Create the appropriate type of component and, in the case of a Database communication point, set the appropriate Mode and Connection Retries settings.

2. 2

Specify the appropriate Message Type and Definition (not required when the communication point is in Input mode).

3. 3

Open the Database Configuration Editor by creating the configuration.

1. Define the Database Connection details.
2. When using the Database Communication Point in Input mode, define the Polling parameters.
3. Check and edit the component Options as necessary.
4. Use the SQL Editor to create the required SQL Statements.
5. Save and Exit from the Editor.

4. 4

Check in the component and test.

Step 3.4, 3. & 4 will generally be repeated as necessary while the configuration is built and tested.

We will explore this process in more detail during the remaining lessons and exercises in this module.

Creating the Configuration

Introduction

In this lesson, we demonstrate the process of creating a simple configuration that writes message fields and properties into a database table.

Component Configuration

Before we start creating the database configuration, there are a few component settings we need to look at. These settings help ensure that our database connection remains

stable and allow us to specify the format of the message processed by the communication point.

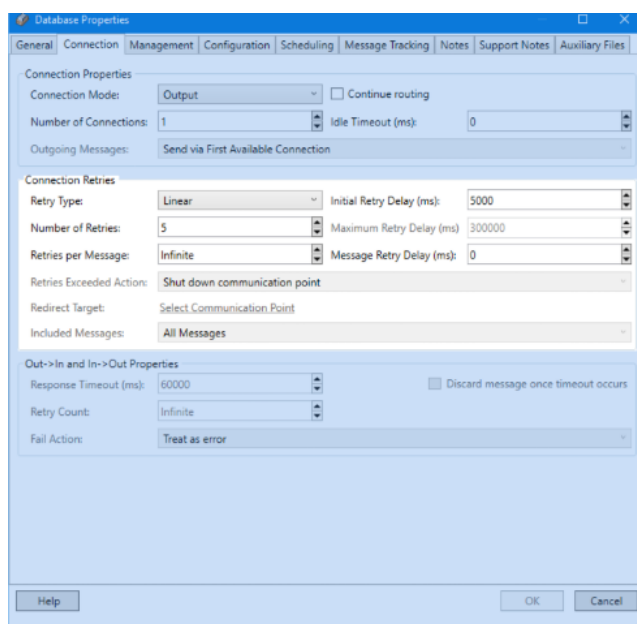
Connection Settings

When configuring a database communication point, we need to consider the appropriate connection retry settings for the database server we are connecting to. The settings that you need for your configuration will depend on the setup of your database server and nature of the messages being processed.

Database Filters

The Database LookUp filter and Database Message Extraction filter cannot use communication point connection retry logic. If a connection failure occurs when using a filter, messages received during the connection outage are sent to the error connector.

Messages that take a long time to process through a database filter can delay other messages on the route, depending on FIFO settings.



Connection Retries Settings

Message Definition

There are two configuration options that let us define how the component will work with messages.

Property	Description
Message Type	By default, Rhapsody will assume that a message contains Plain Text . When this option is selected, we can't use message fields in our database configuration. If the message is XML formatted, we can select XML and use XPath expressions to refer to fields in the message.
Definition	This option overrides the Message Type property by supplying the name of a message definition in Rhapsody.

Timeout Settings

There are two timeout settings that let us control how long Rhapsody will wait for the database to respond.

Property	Value
Query Timeout	The query timeout option specifies how long, in seconds, a database query can take before it times out. The default value of 0 is no timeout. The timeout applies to all SQL queries configured in the component. It is important to set a suitable timeout to ensure that queries do not block longer than expected. Slow queries can create large communication point queues or, in the case of filters, route backlogs that will affect processing across the engine.
Socket Timeout	Specifies how long, in seconds, the underlying connection should remain open before timing out. The socket timeout prevents Rhapsody from waiting indefinitely for results from a database in the event Rhapsody loses its connection to the database. A value of 0 indicates the timeout is not configured.

Database Properties

General Connection Management Configuration Scheduling Message Tracking Notes Support Notes Auxiliary Files

Connection Mode: Output

Properties with an asterisk (*) are required.

Property	Value
Database	Manual setting
Host *	
Port	
Database Name *	
Username	
Password	
Additional Connection Properties	
Configuration File *	Create Configuration
Query Timeout	0
Socket Timeout	0
Message Type	Plain text
Definition	Treat As Message Error
Treat Deadlock as Connection Error	☒
Treat Query Timeout as Connection Error	☒

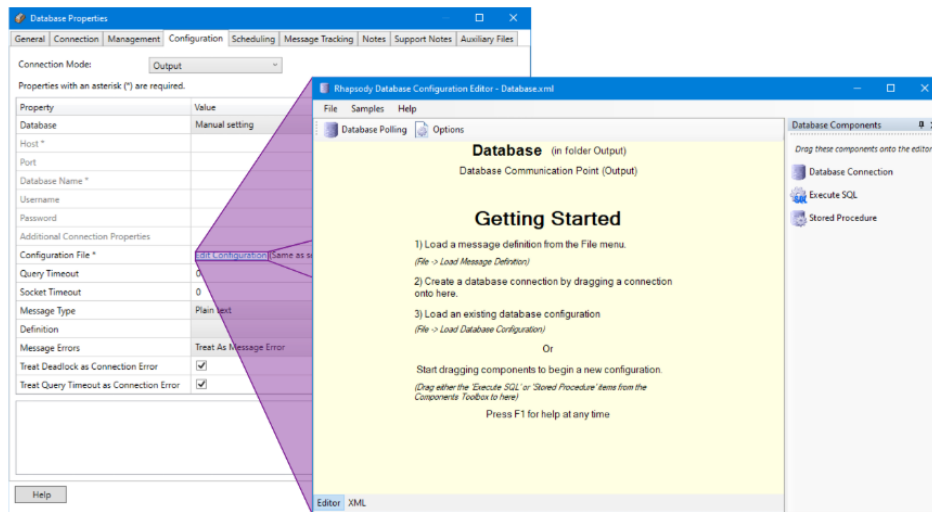
Help OK Cancel

Database Communication Point in Output Mode

The Database Configuration Editor

The **Database Configuration Editor** is a component of the Rhapsody IDE which provides a graphical interface for managing the configurations of the **Database** communication point, Database LookUp filter, and Database Message Extraction filter components.

It is accessed by selecting **Edit Configuration** or, if the database configuration is yet to be created, **Create Configuration** link on the **Configuration** tab of the component. When the Editor opens, the current configuration is loaded for editing. The configuration is saved to the corresponding component when the Configuration Editor is closed or when the **File > Save** option is selected.



Opening the Database Configuration Editor

The Database Configuration Editor includes features to make it useful for a wide range of database tasks. Not all of these features are useful in all situations. We will describe the features required to compete for a particular task as we complete the course.

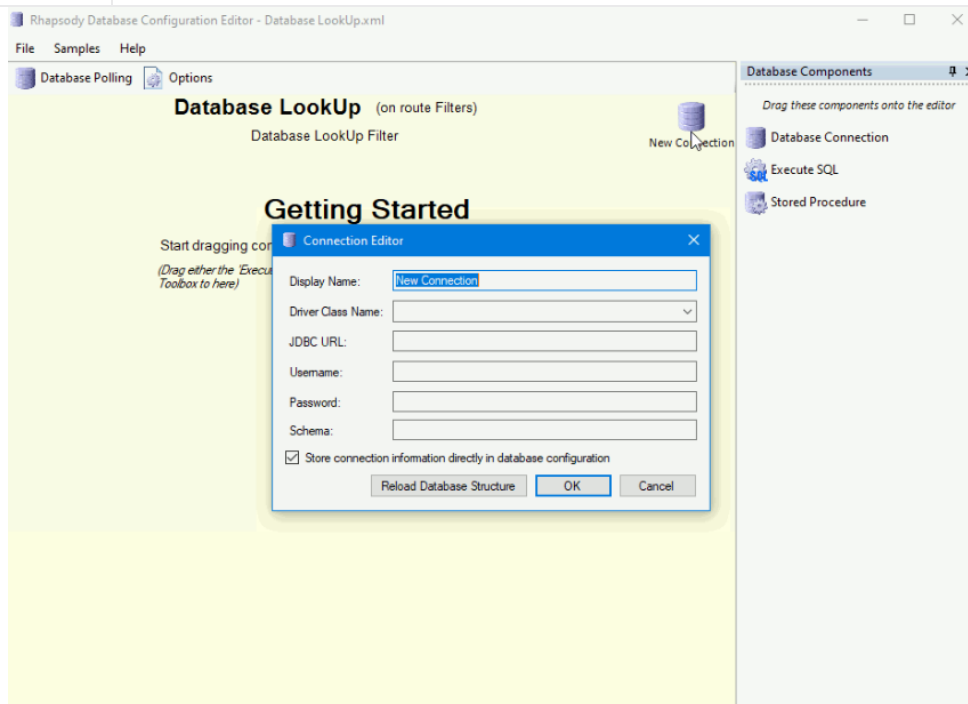
The Database Configuration Editor builds an XML file describing the configuration, which is saved as an Auxiliary File in the component. The current state of the XML can be viewed at any time by selecting the XML tab at the bottom of the Editor. It is generally not necessary to view or edit the XML directly, except in situations where you wish to copy or reorganize large parts of a configuration.

Defining Connection Details

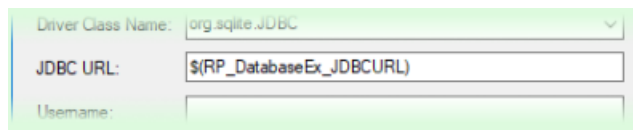
A database connection is added to your configuration by dragging the Database Connection icon onto the title section of the Editor canvas (you can't place it on the body of the canvas, only in the title section). This lets us define the connection settings that we will use to connect to the database server.

The Connection dialog opens as soon as the new connection is added. The following connection details need to be entered:

Property	Description
Display Name	Text that describes the database connection.
Driver Class Name	The name of the JDBC driver that will be used to connect to the database. When using one of the database drivers provided by Rhapsody, the drop-down can be used to select the appropriate value.
JDBC URL	The connection string for the database. The format of this string is database specific. It will always contain the location of the database, but might also include additional database specific information such as connection security settings, or failover details.
Username	The name of the user to connect to the database as (optional depending on the requirements of the database you are connecting to).
Password	The password associated with the database username (optional depending on the requirements of the database you are connecting to).
Schema	The name of the schema in the database to access (optional depending on the requirements of the database you are connecting to).



It is best practice to use Rhapsody Variables in the database connection to define the JDBC URL, Username, Password, and Schema (as required).



The image shows a screenshot of a configuration window with a light green background. It contains three input fields: 'Driver Class Name' with a dropdown menu showing 'org.sqlite.JDBC', 'JDBC URL' with a text box containing '\$(RP_DatabaseEx_JDBCURL)', and 'Username' with an empty text box.

This helps make it easy to migrate your configuration from one environment to another (where it will likely need to be configured to access a different database server).

Once the details have been entered, you can test the connection and load the database schema by selecting the **Reload Database Structure** button.

The connection details need to be valid from the Rhapsody Server. However, when loading the database schema, the Editor uses a connection to the database from the Rhapsody IDE you are using. In some cases, the database may be on a separate network and may not be accessible from your computer. In these cases, either a remote connection must be utilized (for example, using port forwarding from the server), or alternatively, it will not be possible to verify the connection details until the component is checked in to the engine.

Adding a SQL statement

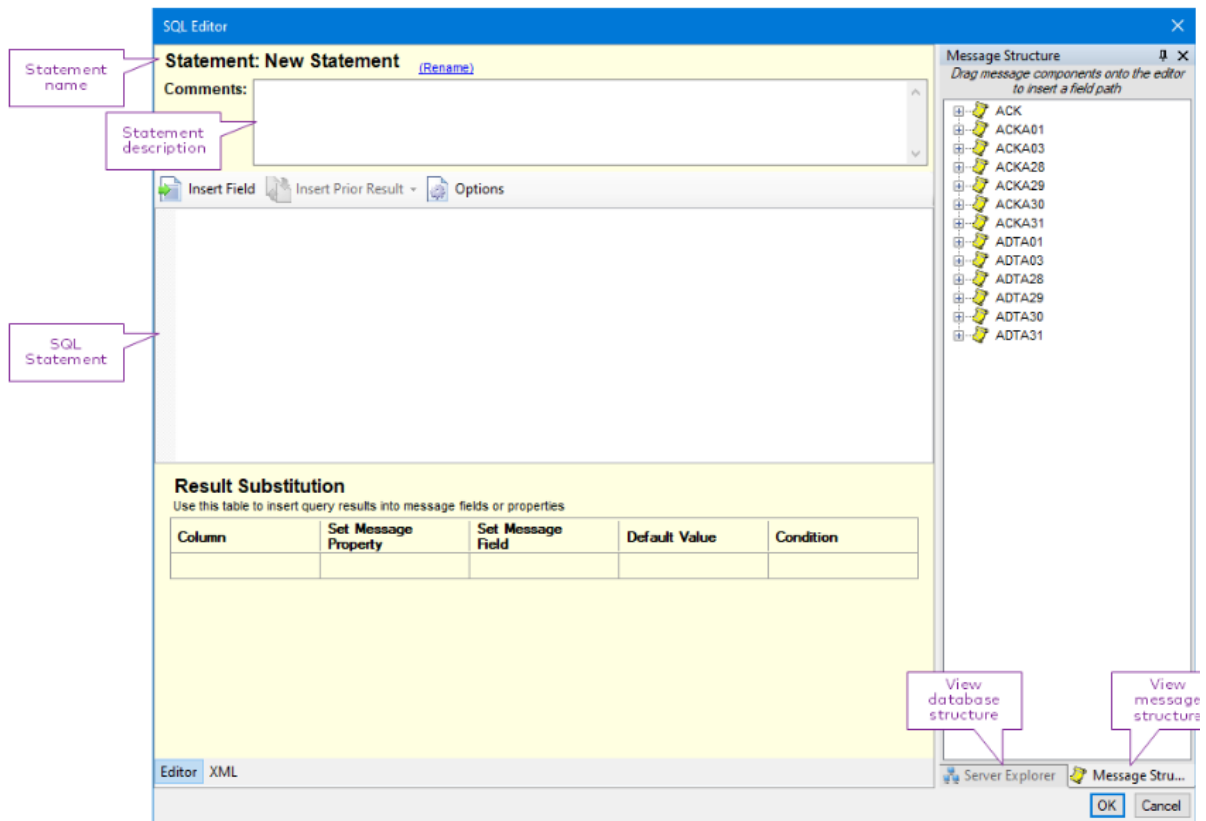
For our database configuration to have an effect, we need to add at least one SQL Statement. This is done by dragging the **Execute SQL** component into your configuration.

Once the new statement has been created, it is edited by double-clicking on the statement name. This will open the SQL Statement Editor window, which will be discussed on the next page of this lesson.

If we need to execute additional queries as part of our configuration, this process can be repeated to add additional SQL statements.

The SQL Editor

The SQL Editor window is where you configure the SQL statements that will be run against the database



SQL Editor Window

Each statement created should have a descriptive name and comments that detail the purpose of the statement, along with anything that may be non-obvious. You can enter the SQL to be run on the server directly into the SQL window, or with the assistance of the SQL Generation functions available from the Server Explorer tab.

When creating complex SQL statements, we recommend that you test the statements directly against the database first to ensure the logic is correct.

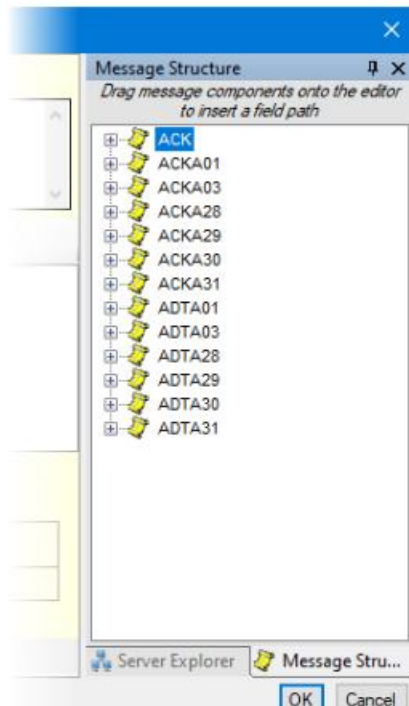
Helper tabs

To the right of the SQL panel are two tabs, Message Definition and Server Explorer. These can be used to inspect the message definition and database structure to help you build the query.

Message Definition

The Message Definition tab displays a tree view of the definition associated with the configuration. If no message definition is associated with the configuration, a note will be displayed in place of the definition.

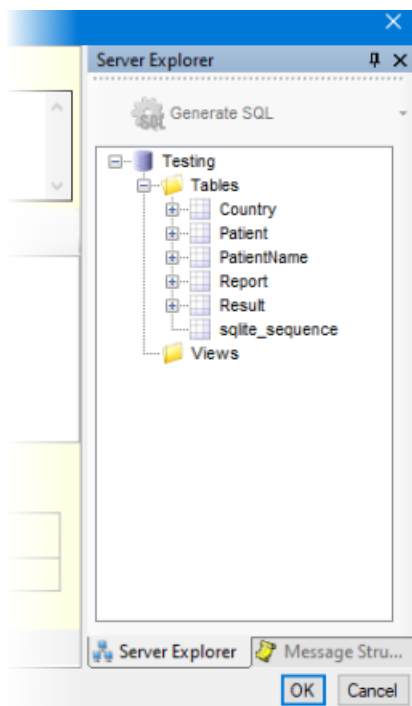
You can copy a field from your message into the SQL by selecting the field to be inserted and dragging it to the appropriate location within the SQL.



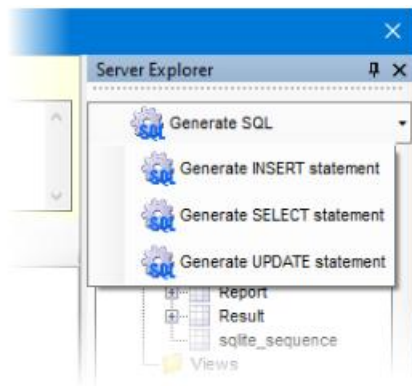
You will sometimes need to manually add the message definition to the database configuration by selecting **File > Load Message Definition** from the main configuration window.

Server Explorer

The Server Explorer tab presents the structure of the database.



When you select a table or view, the **Generate SQL** link at the top of the tab is activated. Selecting this lets you create SELECT, INSERT, and UPDATE statements for the highlighted item.



Generate SQL Links

Adding Message Content to the SQL

When constructing your query, it is common that a part of the message or a message property will be used in the query.

The SQL statement may be populated with values from the message (where the format of the message is known) or from previous statements.

Column 1	Column 2	Column 3
Message Property	\$property_name	\$PatientID
EDI Message Field	@path.to.field	@PID.PatientName[0].FamilyName.Surname
XML Message Field	An XPath expression is used to find the field in the XML message	`/ADT_A01/NK1[1]/NK1.1`
Column from a previous query in the configuration	@column_name	@PatientName
Complete Message Body	\$messageContent	\$messageContent

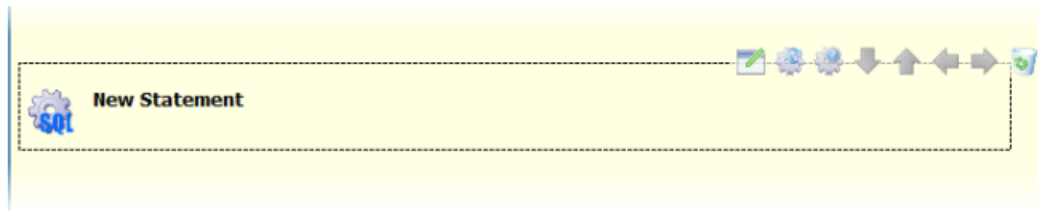
More Complex Query Configuration

Introduction

In the previous lesson, we looked at how we can run a statement against a database to insert content into the database. We will now look at some of the more complex ways we can use databases in Rhapsody. These include updating the message from the database, executing multiple statements, nesting statements, conditional execution, and iterating over parts of a message.

Controlling statement execution

So far, we have used the Execute SQL block to run a statement against a database. For many of the tasks that Rhapsody can perform against a database, we need to be able to control how a statement is executed. When you select a query, a number of buttons appear at the top right corner of the statement.



Statement controls

Below is an explanation for each of the options.



Edit Statement

Opens the statement editor. This is the same as double-clicking the statement.



Configure Statement Repeats

Lets us repeat the statement for all the items of a particular type in the message.



Configure Statement Conditions

Lets us control when the statement is executed (for example: When a previous statement returns a particular value, or when a field exists in the message).



Reorders statements

Controls the order in which statements will be executed (up and down arrows), or groups this statement with other statements (left and right arrows). Only those actions which are possible on the selected statement will be enabled.



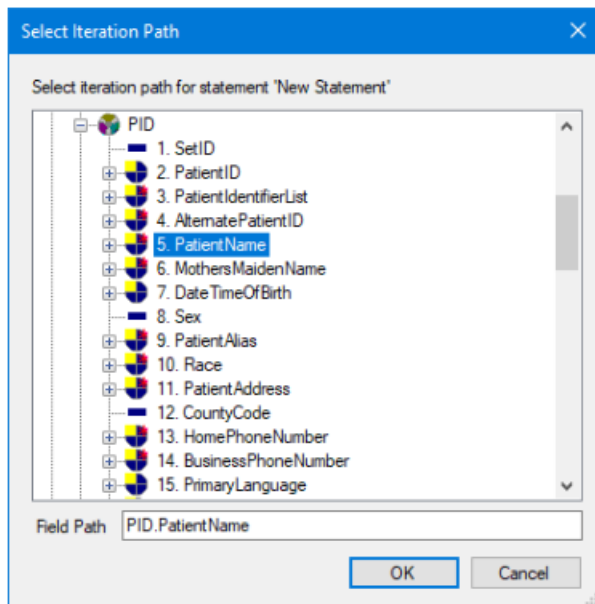
Delete Statement

Removes the statement from the configuration.

Statement repetition

There are times when we need to execute a statement for every repeat of a particular field or set of fields in a message. For example, we might need to execute an INSERT statement for every patient name in a registration message. For repetition to work, either a message definition (EDI or XSD as appropriate) must be associated with the configuration or the **Message Type** property must be set to XML.

To make a query repeat, first select the **Configure Statement Repeats** button for the statement you want to repeat, then select the message node or segment to repeat over. Where the database configuration is using an XML message without a definition, the message path will need to be entered as an XPath expression by hand.



Configuring a statement to repeat on the `PID.PatientName` message path

A statement that has repeats defined on it will display a modified icon that includes a repeat indicator and additional text that indicates what field is being repeated over. Selecting the field allows you to edit the repetition details.



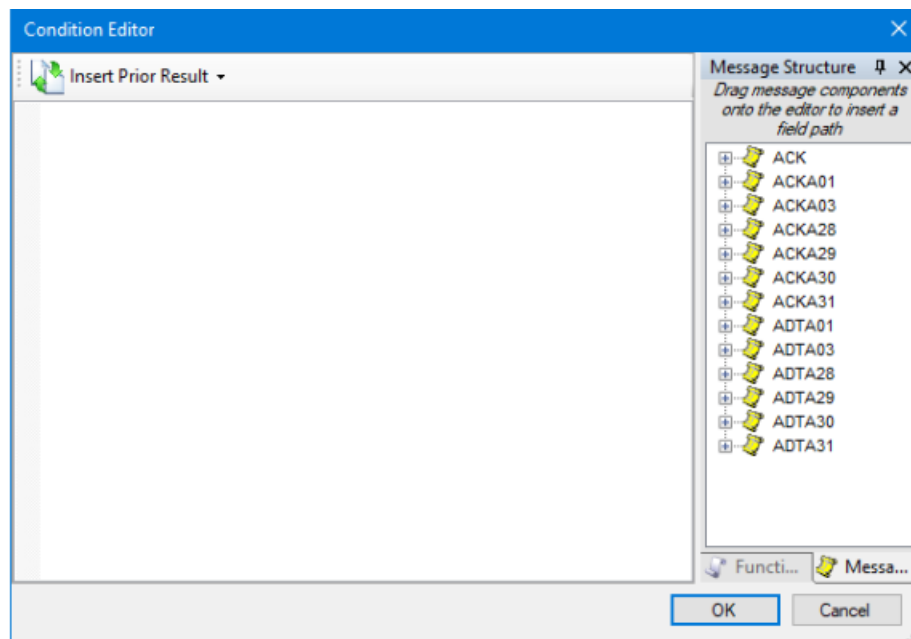
Repeats added to a statement

When constructing a SQL statement that repeats, any message paths that repeat do not need an item index for the element that the statement will iterate over. The easiest way to get correct message path syntax is to select the message node from the **Message Structure** menu on the right of the **SQL Editor** window and drag it to the appropriate place in the SQL statement.

Conditional execution

Conditions can be placed on the execution of queries. These are used to determine if a statement will be executed. All conditions allow message fields (where a message definition exists), message properties, the result of previous queries, and constant values to be checked.

To create a statement condition, select the **Configure Statement Condition** button for the statement you want to control conditions on. This will display the Condition Editor dialog where conditions can be edited



Condition Editor dialog

On the right side of the Condition Editor are two tabs. The **Functions** tab lists all the that can be used, while the **Message Structure** tab displays the structure of the associated message definition. Where there is more than one statement in your database configuration, the **Insert Prior Result** button will become available and can be used to insert a result from a previous statement. This is useful when you need to perform a check against the database before inserting or updating some information.

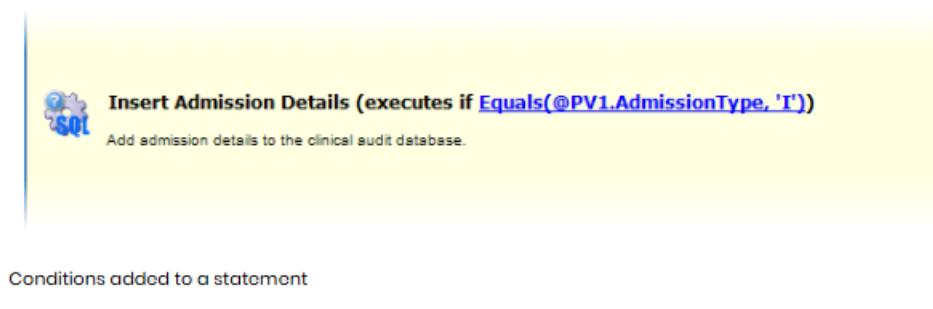
The set of functions that can be used are as follows:

Condition	Description
Equals(param, value)	Compares the two parameters passed to the cond returns a positive result if they match.
EqualsIgnoreCase(param, value)	Works like the Equals() condition, but ignores case "PATIENT" would match "Patient".
NotEqual(param, value)	Compares the two parameters passed to the cond returns a positive result if they do not match.
NotEqualIgnoreCase(param, value)	Works like the NotEqual() condition, but ignores ca

Condition	Description
Null(param)	Inspects the single parameter and returns a positive result if it contains a null string or empty string.
NotNull(param)	Returns the inverse of the Null() condition.

Where a condition is being evaluated against a constant, that constant should be enclosed in single quotes. Other parameter types use the same format as used in a SQL statement.

Similar to repeating statements, a statement that has conditions defined on it will display a modified icon and additional text that indicates the execution conditions. Selecting the conditions allows you to edit the execution rules.



Multiple conditions

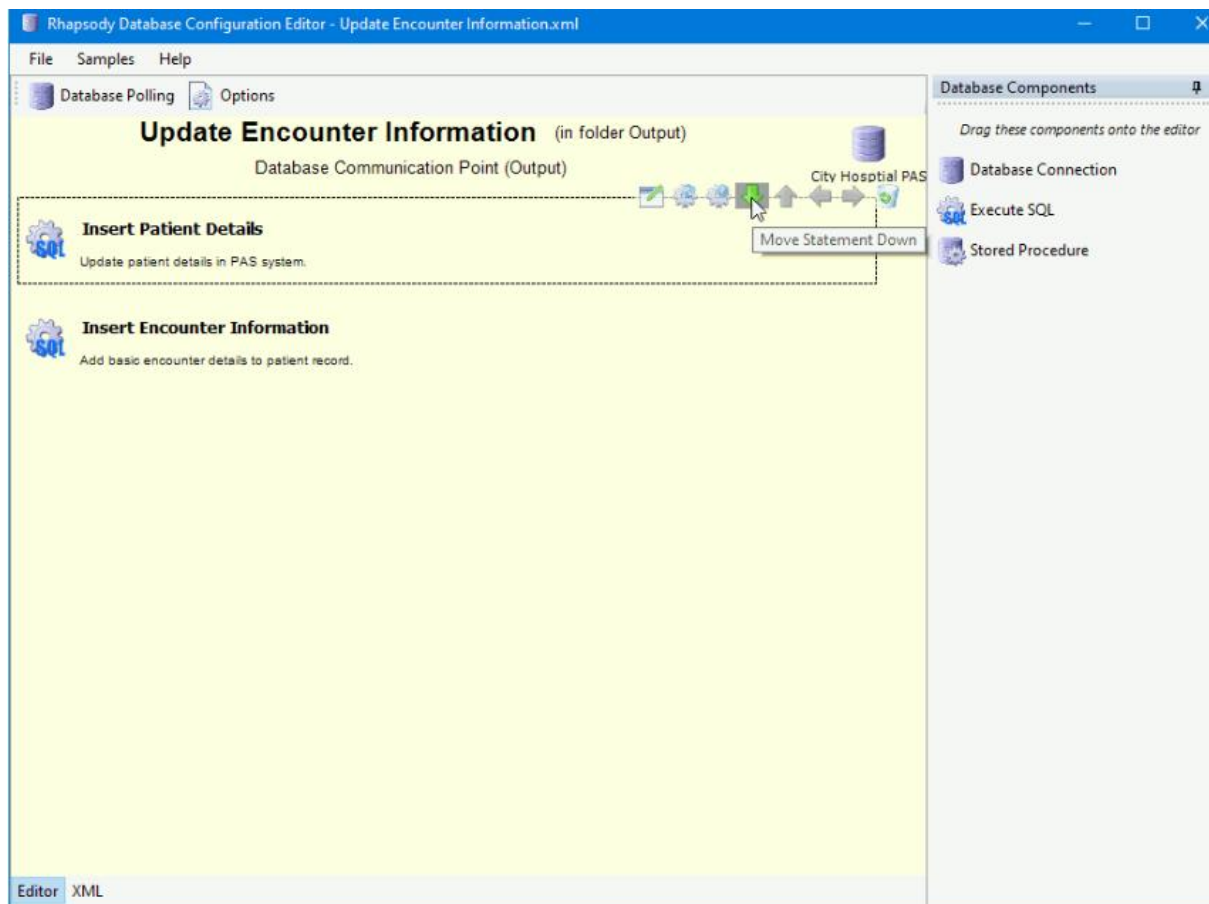
Multiple conditions can be combined using either an and or or keyword where more complex logic is required. Brackets can be used to group conditions.

Equals(@PV1.AdmissionType, 'I') and (NotNull(@PV1.VisitNumber.IDNumber) or NotNull(@PV1.AlternateVisitID.IDNumber))

The example above illustrates all of these being used to construct a condition that will only run the associated statement if the admission type is I and either the visit number or alternate visit number is not null.

Reordering statements

As part of your database configuration, there is frequently a need to run multiple statements against the database. This is achieved by including additional **Execute SQL** components in the configuration. Statements are executed against the database in the order they appear in the configuration. This can be changed by selecting the statement and using the up and down arrows to alter the order.



Animation showing a statement being reordered

Statement Hierarchy

Using the left and right arrows, the hierarchy between statements in the configuration can be changed. When a statement is moved to the right, it becomes a child statement. Any conditions and repetition rules defined on the parent statement will also apply to all associated child statements.

In the following example, the parent statement will repeat on the NK1 segment. The first child statement will repeat for each NextOfKinName field within each NK1 segment in the message, while the second child statement will repeat for each Address field within each NK1 segment in the message. The three statements will only be executed if the StoreNK1 message property has a value of Y.



An example statement hierarchy

Result Substitution

Result Substitution is the mechanism by which messages are updated with the results of a database query. It allows us to update fields and set message properties in a message with the results of each database statement in our configuration. The **Result Substitution** panel can be found at the bottom of the **SQL Editor** window. It can be used with the **Database** communication point in Out->In mode, the Database LookUp filter, or the Database communication point in Output mode (to update message properties when using Continue Routing).

The **Result Substitution** panel is available at all times, but will generate an error at check-in if it cannot be used for the currently configured component and mode. For example, the Database communication point is in Input mode.

The screenshot shows the 'Result Substitution' panel in the SQL Editor. It has a title bar and a subtitle: 'Use this table to insert query results into message fields or properties'. Below the subtitle is a table with five columns: 'Column', 'Set Message Property', 'Set Message Field', 'Default Value', and 'Condition'.

Column	Set Message Property	Set Message Field	Default Value	Condition
PatientId	PatId	@PID.PatientIdentifierL...		<Create Condition>
PatientIdType		@PID.PatientIdentifierL...		<Create Condition>

At the bottom left of the panel, there is a tab labeled 'Editor' and the text 'XML'. At the bottom right, there is a small icon and the text 'Set'.

The Result Substitution panel in SQL Editor

For each result in the statement we want to process, at a minimum we need to specify the **Column** and either the **Set Message Property** or **Set Message Field** values.

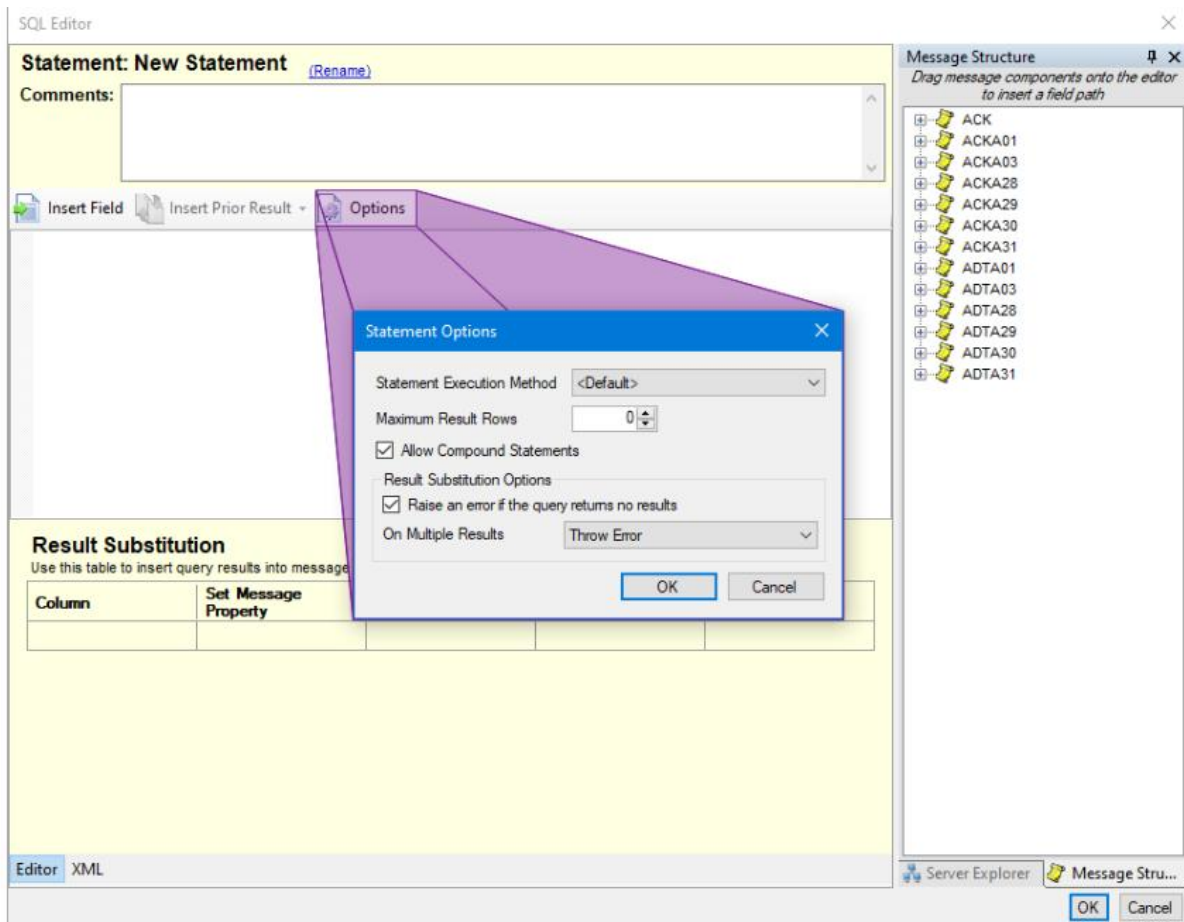
Field	Description
Column	The name of the column returned by the SQL statement to be used in this substitution.
Set Message Property	The name of the message property to set. If the message property already exists, it will be overwritten by the result.
Set Message Field	The field in the message to be updated with the result. Selecting the link in this field will open the Select Message Path dialog. The message must be an XML message or a message definition associated with the database component for this option to function.
Default Value	In the event the statement returns an empty result set, the default value will be used in the substitution.
Condition	Conditions can be used to restrict when the substitution updates the message or properties. Selecting the link will open the Condition Editor. The conditions that may be applied are the same as those used for conditional execution of statements.

The column along with either the message property or message field must be specified for each substitution created.

When working with XML messages, Result Substitution cannot add new fields in the message. In this case, an empty field needs to be added to the XML message before the Database Communication Point. Either the **JavaScript** filter (using the `setField` method) or the **Content Population** filter (using a **Field Mapping**) can be used to create the empty field.

Statement Options

Each statement in your configuration includes a set of options that control how it is executed. Depending on the database server used, the nature of the table or view being accessed, and the query being performed, you might need to alter some options. For most queries, the default options will be fine.



Statement Options Dialog

Option	Description
Statement Execution Method	This option determines how queries are executed against the database. It only needs to be changed from the default value when using some non-standard JDBC drivers.
Maximum Result Rows	This option determines the number of rows returned from a statement where there is support for multiple rows. The default of 0 (zero) indicates that there is no maximum number of result rows.
Allow Compound Statements	This option determines whether Compound Statements can be used in this statement. Compound statements (opens in a new tab) are used to define two or more closely related statements together. Each statement in the Compound Statement is separated with a semicolon. For example, if some values are being inserted into a SQL Server table with an identity column, the INSERT statement inserts the values, and the following SELECT statement retrieves the new identifier.

Result Substitution Options

These options control the behavior of the component when Result Substitution is being used and either no results or more than one result is returned by the SQL query. If an error is raised,

any uncommitted database transactions will be rolled back and the message sent to the error connector for the filter.

Option	Description
Raise an error if the query returns no results	This option determines how queries are executed against the database. It only needs to be changed from the default value when using some non-standard JDBC drivers. This option determines what happens when a statement returns no results. If this option is selected (the default), an error is raised and execution of the remaining statements is cancelled. If the option is not selected, the default values for result substitution are used instead.
Maximum Result RowsOn Multiple Results	This option determines the action to take when the statement returns multiple results, you can select from one of the following modes: • Throw Error (default) - raise an error if a result substitution query returns multiple results. • Read First Result - read the first row, and ignore any subsequent rows. • Read All Results - read all rows. This option adds an Append checkbox to the Result Substitution table. If you select the Append checkbox and define the field to repeat over by setting the repeat to *, all rows will be added to the message. In the example below, each row returned from by the statement will generate a new NK1 segment in the message.

Column	Set Message Property	Set Message Field	Default Value	Condition	Append
givenname		@NK1*1.NextOfKi...		<Create Condition>	☒
middlename		@NK1*1.NextOfKi...		<Create Condition>	☒
surname		@NK1*1.NextOfKi...		<Create Condition>	☒
					☐

Editor XML

Results Substitution showing repeating rows being added to a message

Inserting previous results

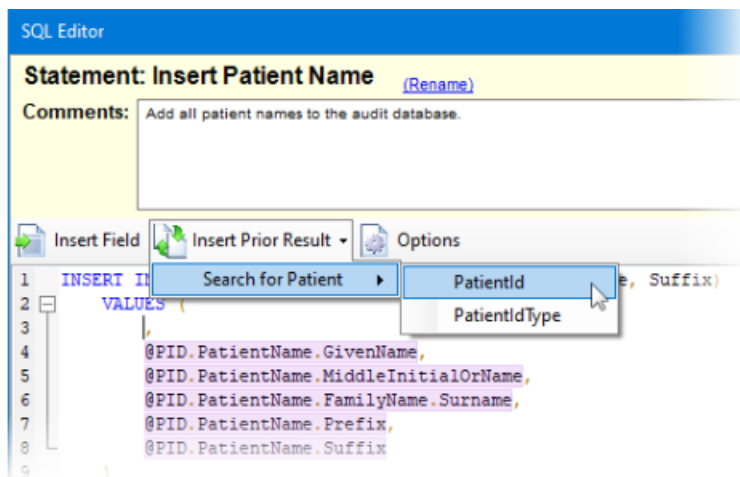
We can insert results from previous statements into statements and conditions. The name of the result to be inserted is taken from the field name as it appears in the SELECT statement. The result name is preceded by a @ character.

In the following example, the PatientId result is being used as part of a condition.

The image shows a 'Condition Editor' dialog box. On the left, there is a list of previous results with 'Insert Prior Result' selected. The main text area contains the condition 'NotNull (@PatientId)'. On the right, a 'Functions' list includes 'Equals', 'EqualsIgnoreCase', 'NotEqual', 'NotEqualIgnoreCase', 'Null', and 'NotNull'. The 'NotNull' function is currently selected. At the bottom, there are 'OK' and 'Cancel' buttons.

A prior result being used in a condition

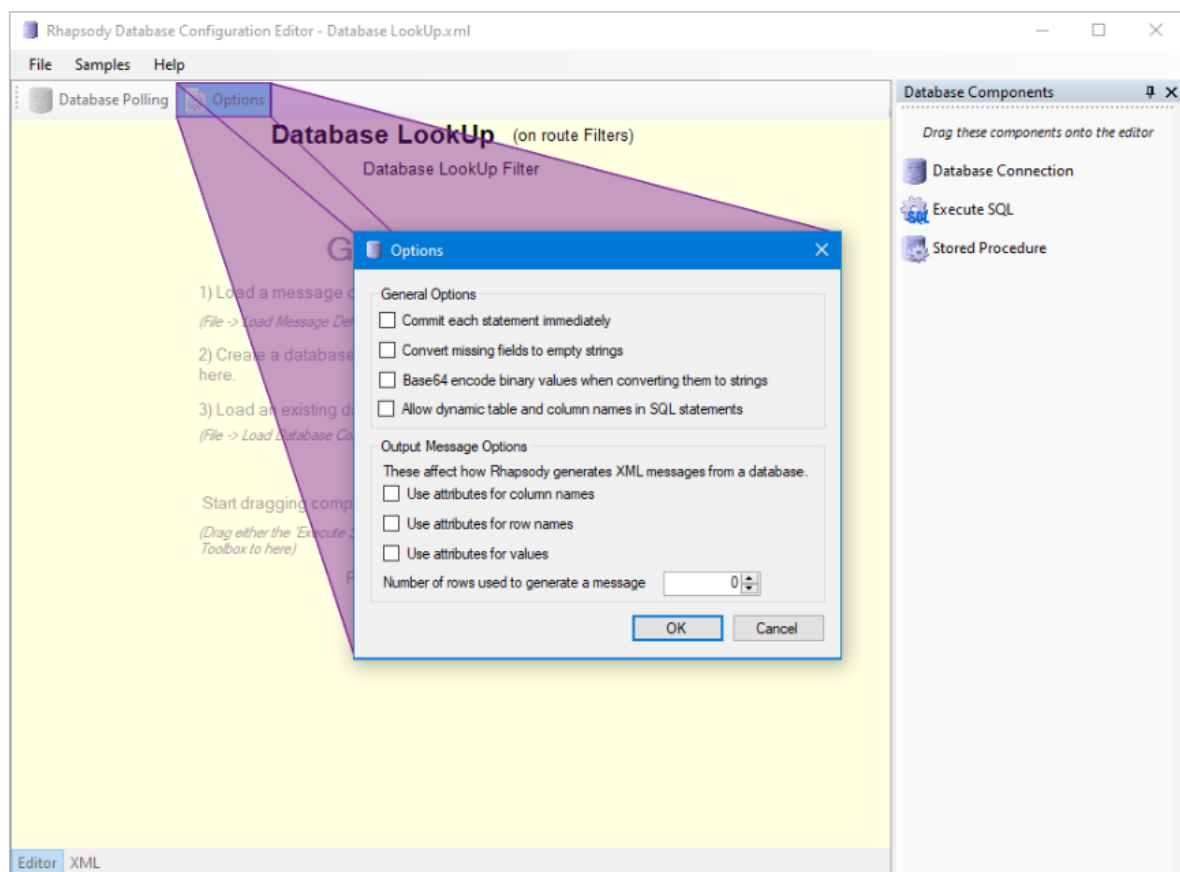
Where the result we want to insert has been used with Results Substitution, the **Insert Prior Result** drop-down can be used to select and insert the result.



A prior result being used in a condition

Configuration Options

There will be occasions when you need to change the execution or output behavior of your database configuration. Make these changes from the **Options** dialogue.



Options dialog

General Options

The default execution mode for the database components is to execute all the configured statements in a single [transaction\(opens in a new tab\)](#), allowing them to be completely rolled

back in the event of a problem. The **Commit each statement immediately** removes that protection and treats each statement as a separate transaction.

Convert missing fields to empty strings ensures that all fields will be returned by the statements with at least an empty string, rather than some fields being returned with a null value. For example, an outer join in a query where the joined table doesn't match any data. This option may be required when subsequent message processing requires all updated message fields and properties to contain a non-null value. Assigning a null value to a message property in a filter will remove the property from the message.

Base64 encode binary values when converting them to strings takes any binary fields in the database and Base64 encode them before inserting the field in a message or message property. If this option is not selected, the field is converted to a string by UTF-8 decoding it. If the binary data does not represent a UTF-8 encoded string, this will likely produce an incorrect value.

Output Message Options

When this configuration is used as part of a Database communication point in Input mode or a Database Message Extraction filter, these options are used to control the format of the XML message generated by the configuration. We will discuss these options in more detail as part of *Generating Messages From the Database* later in this module.

Generating Messages from the Database

Introduction

In this lesson, we look at the two ways we can generate a message from the database. Message generation occurs when we are polling the database looking for changes using the Database communication point in Input mode, or when we want to return large quantities of information from the database using the Database Message Extraction filter. The generated messages are formatted as XML that represents the contents of the result sets returned from the various queries run in the configuration.

Creating a New Message Body

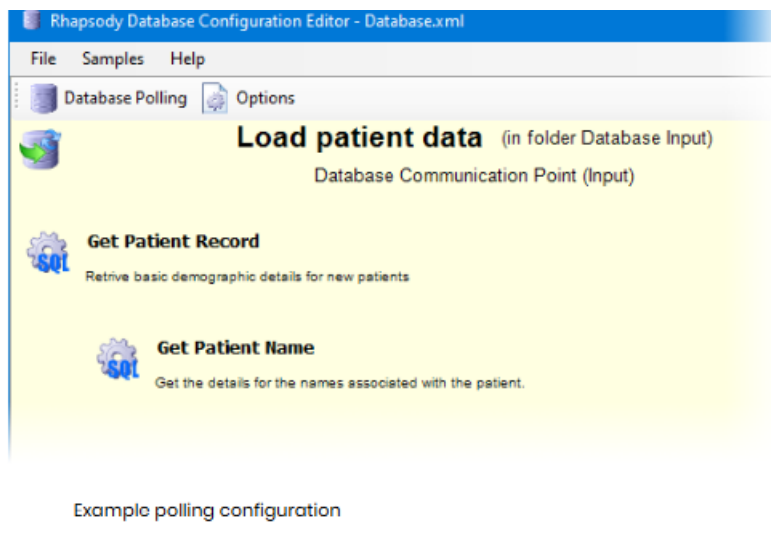
When constructing a message from the database, the result set from each of the SELECT statements is added to the XML message that will be generated. Child statements become child nodes in the generated XML. The name of the node in the XML matches the name of the column returned. INSERT and UPDATE SQL statements can also appear in the configuration, although they won't affect the generated XML. This could be useful where we need to document which records have been retrieved from the database.

Where messages are being constructed as part of a Database communication point running in Input mode, Polling parameters (discussed on the next page) should be configured.

When using the Database Message Extraction filter, we can use fields and properties from the input message in the same way we have done with other components to control the records to be returned from the database.

Example

In the following example, we have a configuration consisting of two statements. The first returns patient details. A child statement gets all the patient names associated with each patient record. Following it is some XML that might be generated by this configuration. Notice how it is possible to get multiple names for each patient record.



```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<message>
```

```
  <GetPatientRecord>
```

```
    <id>8207566</id>
```

```
    <idType>MRN</idType>
```

```
    <gender>F</gender>
```

```
  <GetPatientName>
```

```
    <givenName>Maxine</givenName>
```

```
    <middleName>Anne</middleName>
```

```
    <familyName>Jones</familyName>
```

```
    <current>Y</current>
```

```
  </GetPatientName>
```

```
  <GetPatientName>
```

```
    <givenName>Maxine</givenName>
```

```

        <middleName>Anne</middleName>

        <familyName>Crocker</familyName>

        <current>N</current>

    </GetPatientName>
</GetPatientRecord>
<GetPatientRecord>

    <id>9724875</id>

    <idType>MRN</idType>

    <gender>M</gender>

    <GetPatientName>

        <givenName>John</givenName>

        <middleName>Ian</middleName>

        <familyName>Mooseson</familyName>

        <current>Y</current>

    </GetPatientName>

</GetPatientRecord>
</message>

```

Using the Generated XML Message

A generated XML message may be used the same as any other XML message in subsequent filters and communication points in Rhapsody. The **XSD Generator** will sometimes be used to generate a XSD for the XML so that the mapper can be used to map the generated XML message. At other times, a JavaScript filter may be used to process the message.

Where a generated XML message contains multiple records and records need to be processed individually, the Batch / De-batch filter can be used to split the message into one message per record. The following Batch / De-batch filter configuration could be used with the XML generated in the previous example to separate the incoming XML into individual messages.

De-batch database reports Properties

General Configuration Message Collector Notes Support Notes Auxiliary Files

Properties with an asterisk (*) are required. [Test Filter Configuration](#)

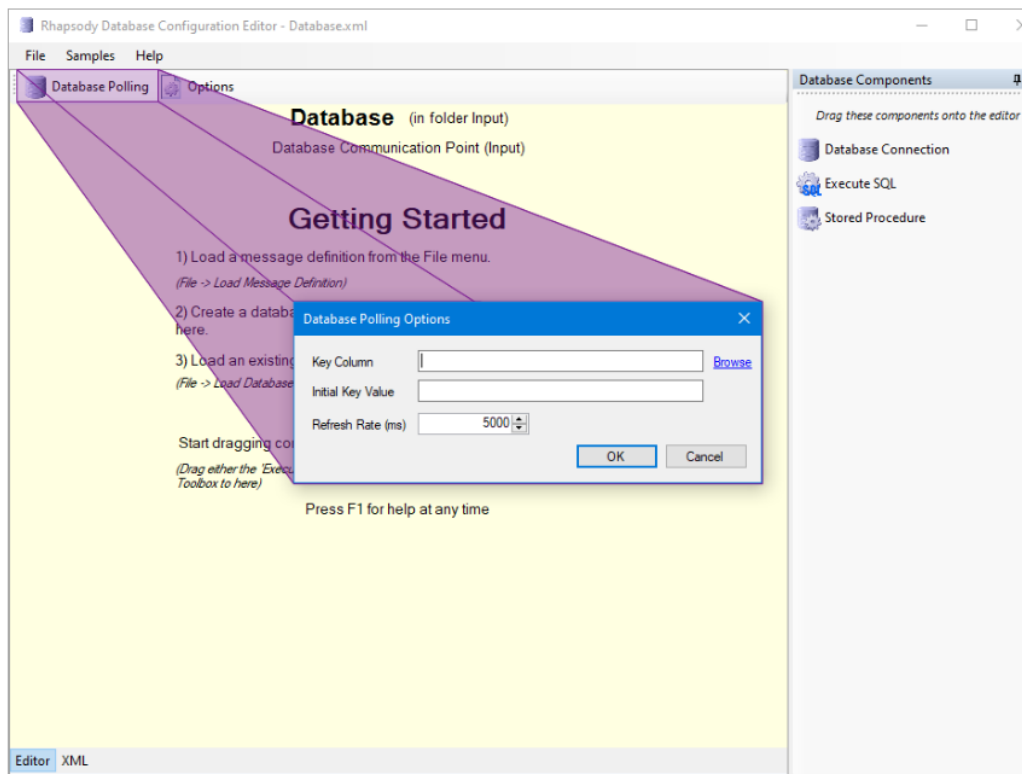
Property	Value
Mode *	De-batch
Header	
Separator	
Trailer	
Batch Header Expression	<message>
Batch Trailer Expression	</message>
Separator Type *	Separates messages
Message Separator Expression *	<GetPatientRecord>
Set Identifier Property	
Index Base *	Zero

Help OK Cancel

Polling Parameters

When using the Database communication point in Input mode, we need to be able to restrict results returned by the statements in the configuration to just new records. If we didn't restrict the results returned, the communication point would continually flood the engine with the same set of messages. This restriction is normally implemented by specifying a column in the first statement we execute that we will filter against. The contents of this column need to be either strictly ascending or strictly descending. Generally, it will either be a timestamp or a database key, often implemented as an auto-incrementing field.

The **Database Polling Options** dialog is used to configure the field we will use to filter the results.



Database Polling Options

Property	Description
Key Column	The name of the column in the statement result set to be used for filtering.
Initial Key Value	The initial value of the Key Column to be used before the statement has run, and a value has been established for it.
Refresh Rate	When the statement returns no results, this determines how long Rhapsody will wait before rerunning the statements. If the statement returns some results, it will be rerun immediately.

Using the Key Column

The key column is inserted into your SQL as if it were a prior result. In the following example, A key column called `modified_date` has been configured and added to the `WHERE` clause in the SQL statement.

```
SELECT
```

```
    patientID,
```

family_name,
given_name,
status,
modified_date

FROM PatientDetails

WHERE modified_date > @modified_date

As the query is configured, it will return all results in the database with a modified_date that is greater than the value of modified_date last time the query was run.

Managing Database Key Values

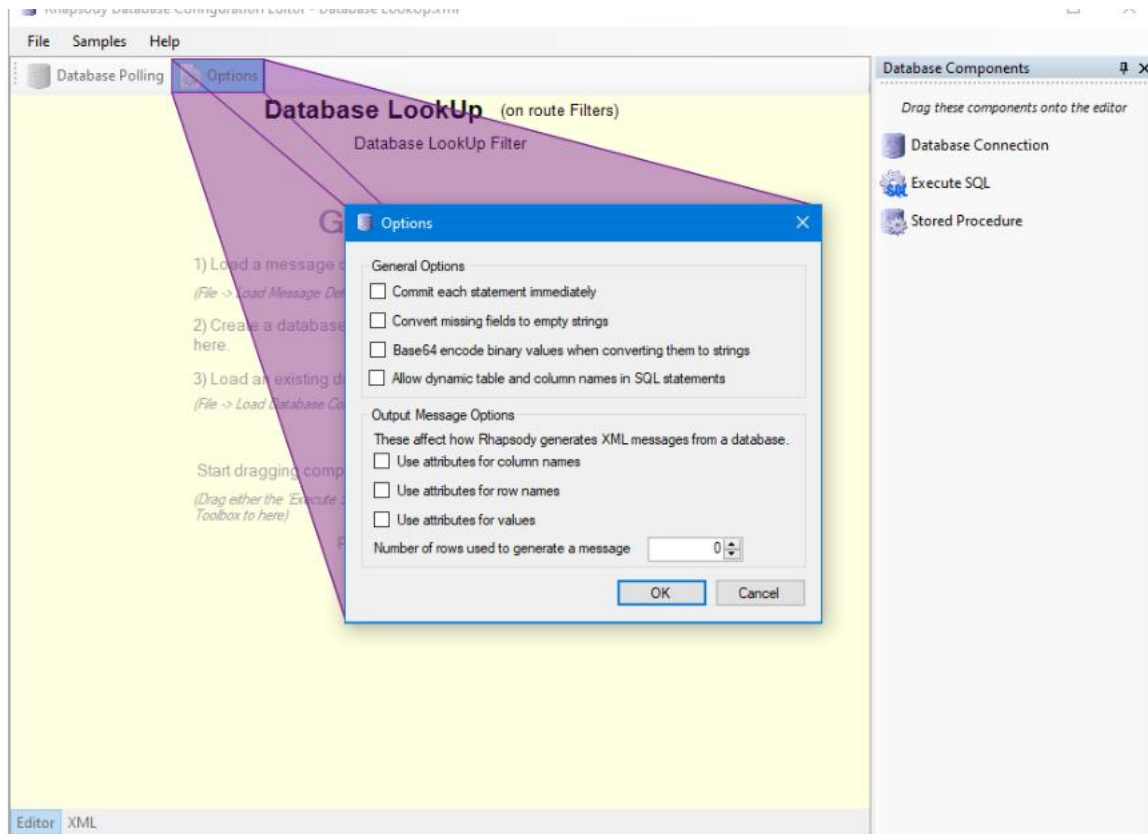
The current value of all database keys is viewable from the **System State > Database Key Values** screen in the **Management Console**. You can update entries on this screen for any stopped communication point by selecting **Edit** for a given entry. This may be useful if you need to reprocess some records in the database, or during testing when we want message processing to start at a specific point.

Name ^	Path	Current Value
Get Records from DB	Finance/Reconciliation	50
Load patient data	PAS/Registration/Backload	2018-06-25 06:26:17 Edit

Save All | Cancel All

Configuration Options

The **Options** dialog contains a section for **Output Message Options**. This allows us to control the format of the XML message generated.



Database Configuration Options

The **Number of rows used to generate a message** option controls the number of records returned in each message. Generally, this is left as the default value of 0, which means that a single XML message is generated for all the new rows in the table. Changing this value limits the number of rows that will be included in each XML message. For example, setting this value to 1 causes a separate XML message to be generated for each row. Subsequent polling of the database will return any additional rows that exist in the table.

Database Inserting Communication Point

Introduction

The Database Insertion communication point operates in output mode and is used to add or update columns in a single database table with information extracted from an incoming message. This functionality makes it a useful archiving tool. This communication point can be used when a single database table needs to be updated and the extra complexity of the Database communication point is not required.

In all cases it is possible to use the database communication point in place of the Database Insertion communication point.

Database Insertion Properties

General | Connection | Management | Configuration | Scheduling | Message Tracking | Notes | Support Notes | Auxiliary Files

Connection Mode: Output

Properties with an asterisk (*) are required.

Property	Value
Definition	Academy-HL7v2.4-ORU.s3d (Same as server)
Database	Manual setting
Host *	
Port	
Database Name *	
JDBC Driver *	org.sqlite.JDBC
Database URL *	\$(RP_DatabaseEx_JDBCURL)
Username	
Password	
Additional Connection Properties	
Database Table *	Messages
Column Values *	[MsgCtrlId,MSH.MessageControlID,Field],[SendingFacility,MSH.SendingFacility
Mode *	Insert
Update Where Clause	
Query Timeout	0
Socket Timeout	0
Treat Deadlock as Connection Error	☐
Treat Query Timeout as Connection Error	☐

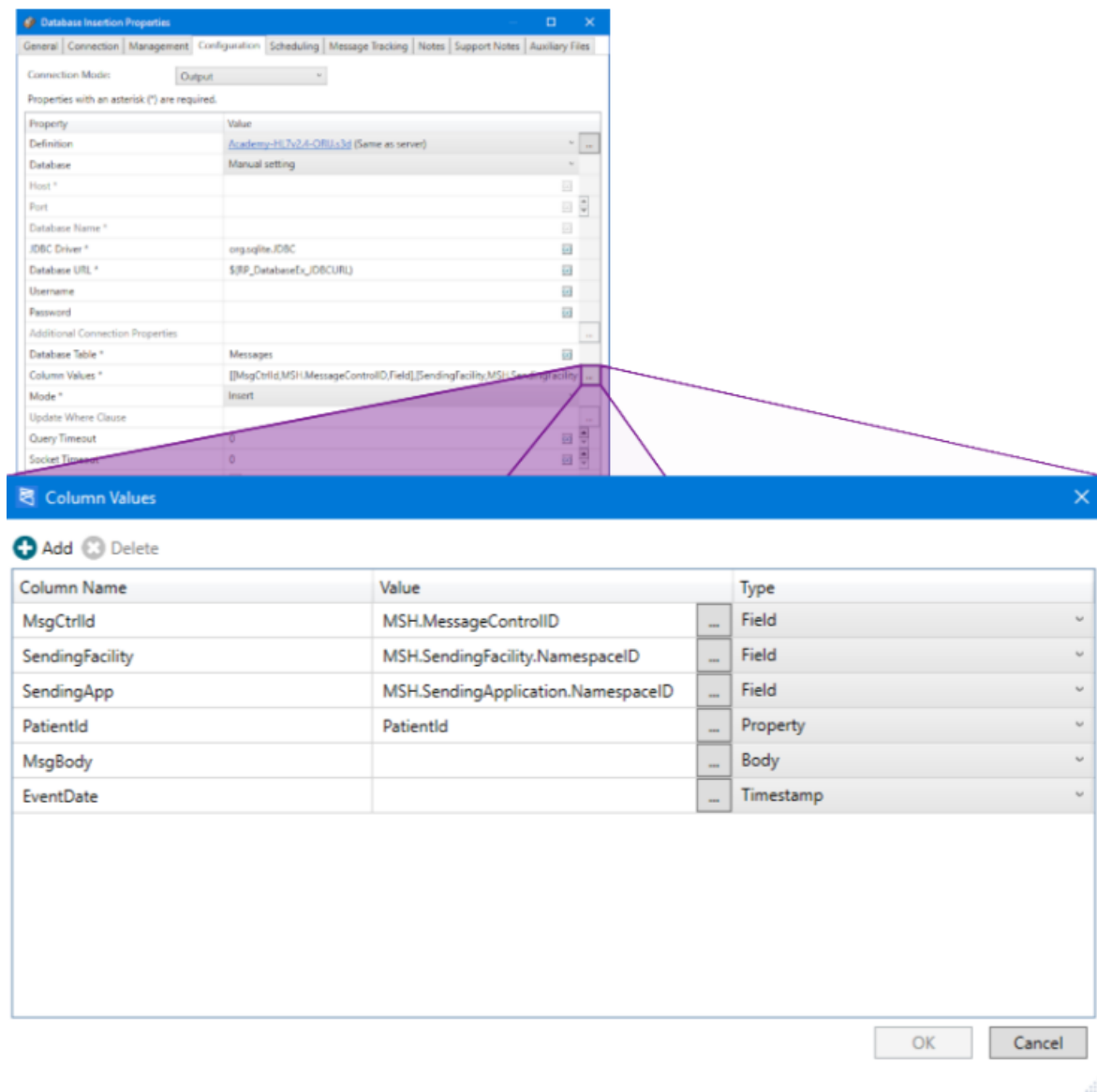
Help OK Cancel

base Insertion communication point configuration properties

Property	Description
Definition	The message definition that identifies the structure and type of incoming messages.
Database (and following properties)	Connection details to the database holding the table whose columns are to be populated or updated with information extracted from incoming messages.
Database Table	The name of the database table that will be updated.
Column Values	Identifies the columns to be updated, along with the source of the information. We will look at this property in more detail in the next page of this lesson.
Mode	Selects between an Insert (a new row is added to the table in the database) and an Update (an existing row in the database is updated) operation.
Update Where Clause (only available when Mode is set to Update)	Defines the conditions under which information in the database table will be updated. We will look at this property in more detail later in this lesson.

Column Values

The **Column Values** property identifies which columns in the database table are to be populated or updated, along with where that information is to come from.



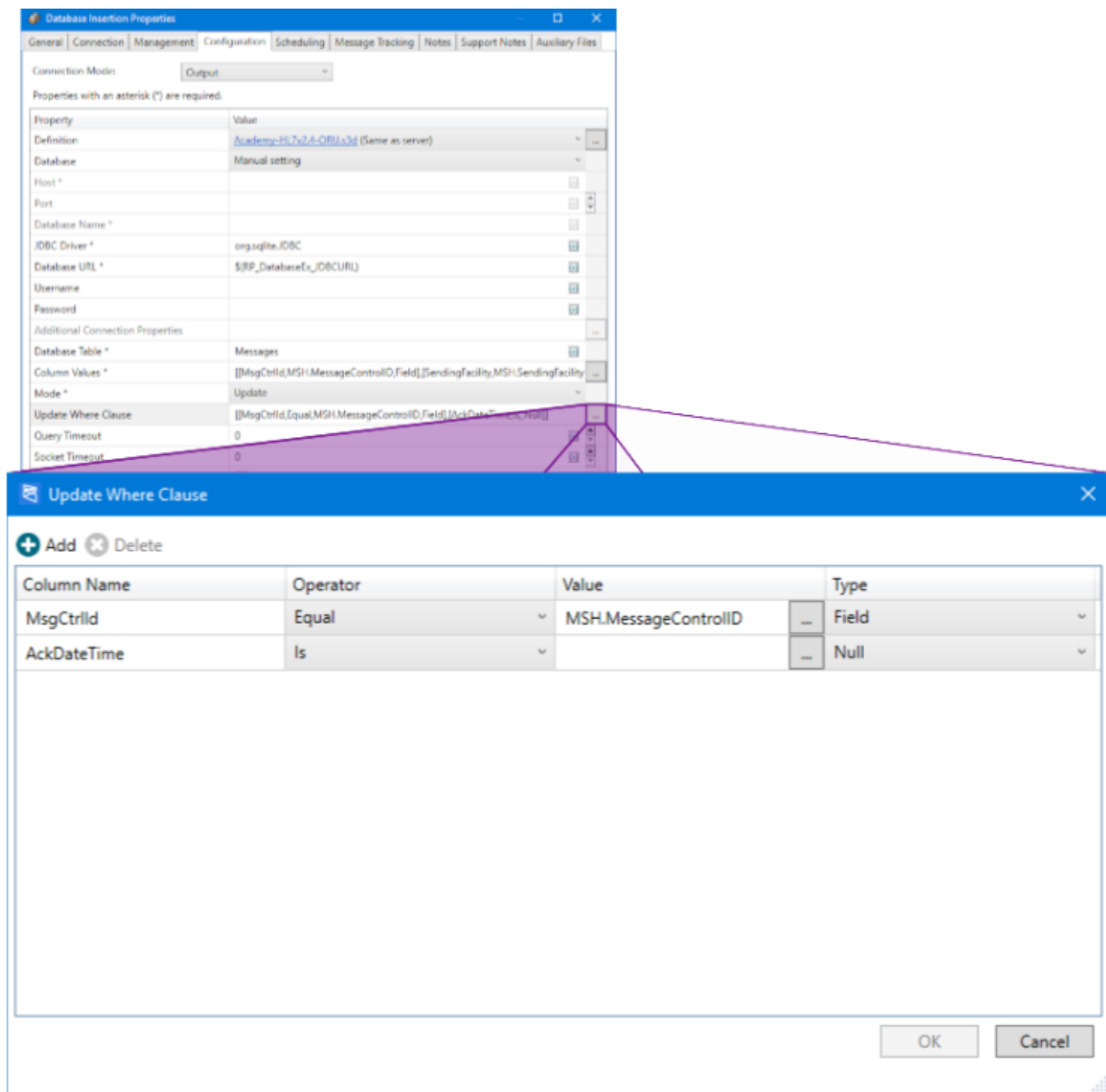
Column Values dialog box

The **Type** column determines the source of the data to be inserted in the database field. The possible values are:

Type	Description
Null	Inserts null into the matching database column. If selected, content in the Value column is ignored.
Body	Inserts the full message body as a byte array into the database column.
Timestamp	Inserts the current date/time as a timestamp into the database column. If selected, content in the Value column is ignored.
Literal	Inserts the content of the Value column as a string into the matching database column.
Field	Inserts the value of the field in the incoming message specified by the Value column.
Property	Inserts the value of the message property specified by the Value column.

Update Where Clause

The **Update Where Clause** property is used when the communication point is configured in Update mode to update an existing record in the database. It identifies any conditions that must be met before existing information in the database table will be updated with data from an incoming message.



Update Where Clause example

In the example above, an existing row will only be updated if the value in the MSH.MessageControlID field of the incoming message matches the value in the MsgCtrlId column in the database table and the AckDateTime database column is null.